Tokenization - In LLM

Breaking raw input (text) into smaller pieces called **Tokens.**

→ Model don't read text
They read TOKENS → converted to numbers → embeddings

Text → Codingdidi is a girl!

→ Tokens :-
["Coding"] ["didi"] ["is"] ["a"] ["girl"] ["!"]

Notice:
- Spaces Matter
- Word may be split.
- Tokens are not always full words.

Why Tokenization?

1. Convert text → manageable unit
2. Handle unknown words
3. Reduce memory usage
4. Enable generalization

Types of Tokenization

① Word Tokenization

⇓

splits by words.

"I Love AI"

[I] [LOVE] [AI]

- Huge Vocabulary
- Unknown words break system
- Doesn't handle Typos/ slang well.

② Character Tokenization

⇓

split by characters

⇓

"I Love AI"

[I] [L] [o] [v] [e] ....

Pros

↳ No unknown words.

Cons

↳ Very long sequences

↳ Harder to learn meaning.

③ Subword Tokenization

⇓

split into meaningful chunks.

"Unhappiness"

[UN] [happi] [ness]

- Handles unknown words.
- Keeps Vocabulary manageable.
- preserves meaning.

Subword Tokenization is used in modern LLMs (BPE, Word Piece, sentence - Piece etc.)

Popular Algorithm.

1. BPE (Byte pair Encoding)
   - starts with characters
   - Merge most frequent pairs
   - Used in early GPT → GPT-2

2. Word Piece.
   - Build Tokens based on probability.
   - keeps most useful subword.
   - Example — playing — [Play] [##ing]
   - Used in BERT

3. Sentence piece.
   - Treats Text as raw stream
   - Doesn't rely on spaces.
   - Used in T5, LLaMa.

4. Unigram Language Model.
   - chooses most probable token set.
   - Most flexible than BPE

What happens after Tokenization

Text ⟶ Tokens ⟶ Token ID ⟶ Embedding ⟶ Model.

"Hello" ⟶ [Hel] [lo] ⟶ [1542, 876] ⟶ [vector1, vector2] .....  ....

** Tokenization affects everything

Context length

Models don't see words.

they see Tokens.

"1000 Words ≠ 1000 Tokens

1 token ≈ 0.75 Word.

Cost

APIs charge per token.

More Token = More Cost

Performance

Bad Tokenization

→ breaks meaning

→ hurts accuracy.

Good Tokenization

→ better understanding

→ better generation